



Testes com Mocks

Testes com Mocks

Como testar um objeto que depende de um recurso caro ou complicado?

→ Crie uma versão fake do recurso que responda com constantes.

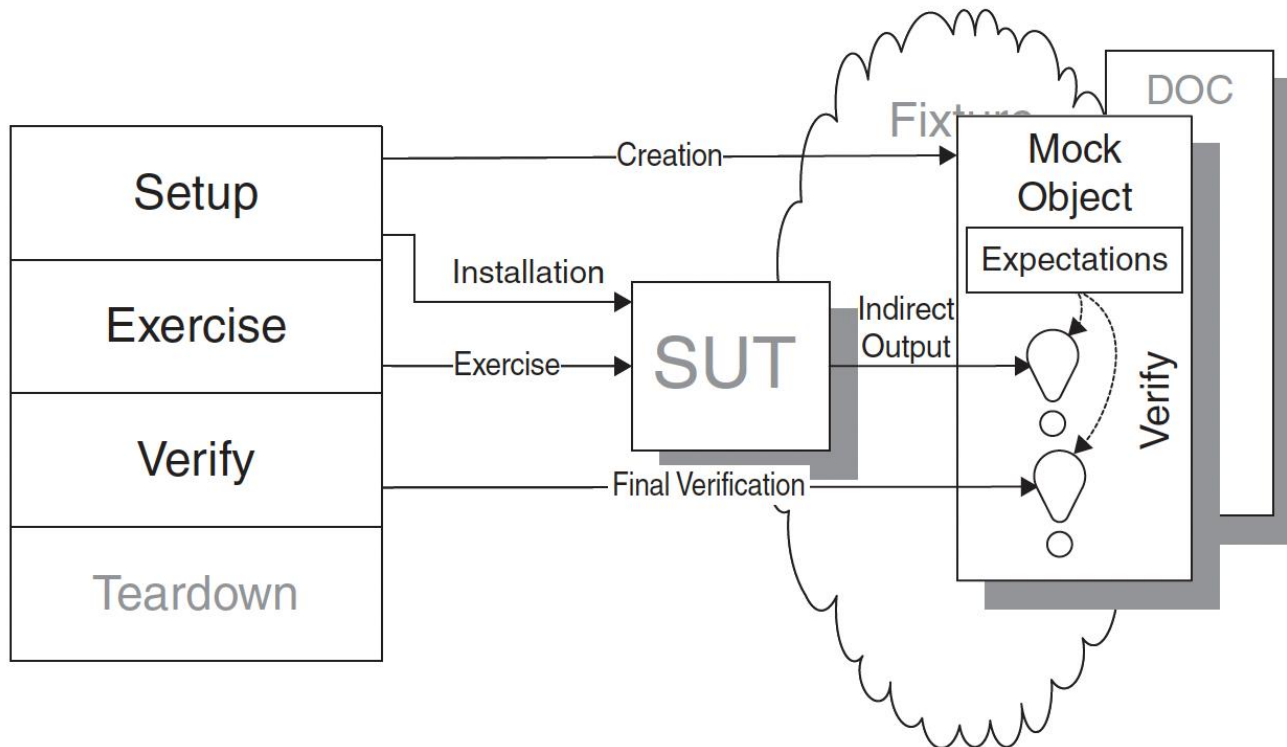
Exemplo: banco de dados.

Testes com Mocks

- **Exemplo:** banco de dados.
- Os bancos de dados demoram muito para iniciar, são difíceis de serem mantidos “clean” e, se estiverem localizados em um servidor remoto, associam seus testes a um local físico em uma rede. Os bancos de dados também são fontes férteis de erros no desenvolvimento.
- A solução é, em vários testes, não usar um banco de dados real. Estes testes são escritos em termos de um objeto que atua como um banco de dados, mas na verdade não é.

Testes com Mocks

- Um objeto do qual o SUT depende é substituído por um objeto específico de teste (Mock) que é chamado quando o SUT é exercitado.



Testes com Mocks em Python

- **unittest.mock** é uma biblioteca para testes em Python que permite substituir partes do SUT por objetos simulados (mocks) e fazer asserts sobre eles.

Exemplo SEM mock em Python

```
def test_change_tiles_1_2_e_2_2_in_board_of_2x2(self):
    board = Board(2, 2)
    board.put_tile(1, 1, 1)
    board.put_tile(2, 1, 2)
    board.put_tile(3, 2, 1)
    board.put_tile(None, 2, 2)
    board.change_tiles_in_positions(1, 2, 2, 2)
    self.assertEqual(1, board.get_tile(1, 1))
    self.assertEqual(None, board.get_tile(1, 2))
    self.assertEqual(3, board.get_tile(2, 1))
    self.assertEqual(2, board.get_tile(2, 2))
```

Exemplo de sintaxe COM mock em Python

```
from unittest.mock import patch, Mock
```

o método `get_tile` está sendo mockeado

```
@patch('board.Board.get_tile')
```

```
def test_change_tiles_1_2_e_2_2_in_board_of_2x2(self, mock_board_get_tile):
```

```
    board = Board(2, 2)
```

```
    board.put_tile(1, 1, 1)
```

```
    board.put_tile(2, 1, 2)
```

```
    board.put_tile(3, 2, 1)
```

```
    board.put_tile(None, 2, 2)
```

```
    board.change_tiles_in_positions(1, 2, 2, 2)
```

```
    mock_board_get_tile.return_value = 1
```

```
    self.assertEqual(1, board.get_tile(1, 1))
```

```
    mock_board_get_tile.return_value = None
```

```
    self.assertEqual(None, board.get_tile(1, 2))
```

```
    mock_board_get_tile.return_value = 3
```

```
    self.assertEqual(3, board.get_tile(2, 1))
```

```
    mock_board_get_tile.return_value = 2
```

```
    self.assertEqual(2, board.get_tile(2, 2))
```

{ prestar atenção
somente na sintaxe...
e não no contexto }

valor que o método mockeado vai retornar quando for chamado

Exemplo SEM mock em Python

```
def test_change_tiles_in_invalid_position_1_3_e_1_1_in_board_of_2x2
(self):
    board = Board(2, 2)
    board.put_tile(1, 1, 1)
    board.put_tile(2, 1, 2)
    board.put_tile(3, 2, 1)
    board.put_tile(None, 2, 2)
    with self.assertRaises(Exception):
        board.change_tiles_in_positions(1, 3, 1, 1)
```


Exemplo de sintaxe COM mock em Python

```
from unittest.mock import patch, Mock
```

```
@patch('board.Board.change_tiles_in_positions')
```

```
def test_change_tiles_in_invalid_position_1_3_e_1_1_in_board_of_2x2  
    (self, mock_change_tiles_in_positions):
```

```
    board = Board(2, 2)
```

```
    board.put_tile(1, 1, 1)
```

```
    board.put_tile(2, 1, 2)
```

```
    board.put_tile(3, 2, 1)
```

```
    board.put_tile(None, 2, 2)
```

```
    mock_change_tiles_in_positions.side_effect =
```

```
        Mock(side_effect=InvalidPositionException())
```

```
    with self.assertRaises(Exception):
```

```
        board.change_tiles_in_positions(1, 3, 1, 1)
```

{ prestar atenção
somente na sintaxe...
e não no contexto }



o método **change_tiles_in_positions**
está sendo mockeado